The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software
Unit Coverage: 1-5
Student Name: Ammar Khan
Registration Number: 24BCY10236
Slot: D11
Date of Submission: 31st March, 2026

1. Introduction

We live in an age where software is no longer a luxury — it is the infrastructure of civilization. The servers hosting websites, the tools powering daily development workflows, and the operating systems running everything from supercomputers to smartphones were not engineered behind closed doors by a single corporation seeking profit. They were built in the open, shared without restriction, and refined collectively by individuals who believed that progress belongs to everyone. This is what open source means at its core.

This project moves beyond theory. Rather than discussing open source in the abstract, it examines the real friction, necessity, and human decisions that give birth to a major open-source project — the compromises made, the crises endured, and the breakthroughs that followed.

The software chosen for this audit is Git — the version control system that underpins virtually all modern software development. This report does not simply describe what Git does. It investigates why Git had to exist: the frustration that sparked it, the individual who built it, the legal freedoms its GNU General Public License actually guarantees, the philosophical convictions it embodies, and the profound and lasting ways it has reshaped how developers and creators around the world collaborate on code.

Alongside this written analysis, five shell scripts have been developed to demonstrate practical Linux skills. These scripts are not academic exercises — they show how open-source tools behave at the system level, and how the principles of automation and transparency translate from philosophy into working, executable code.

Every line of code written today stands on an open-source foundation. Understanding the thinking behind that foundation is not supplementary to a technical education — it is the foundation itself.

2. Choosing Your Software

For this audit, I have selected the following software from the approved list:

- **Software:** Git
- **Category:** Version Control
- **License:** GPL v2 (GNU General Public License version 2.0)
- **Why it's interesting:** It is the tool Linus Torvalds built over a single weekend when a proprietary software company revoked his community's free license, thereby, permanently changing how the world writes code and how private software companies/organizations work.

3. Project Report Structure

PART A: Origin and Philosophy

A.1 The problem this software was created to solve

Every great piece of software has a moment of origin — a specific frustration, a specific failure, a specific wall that someone hit and refused to accept. For Git, that moment was not a gradual dissatisfaction with existing tools. It was a single phone call in April 2005 that left thousands of developers with nothing.

But to understand that moment, you have to start much earlier.

1991 — A Student, a Kernel, and No Tools

Linus Torvalds was twenty-one years old and studying computer science at the University of Helsinki when he began writing what would become the Linux kernel. He was not trying to start a movement. He was trying to solve a personal problem: he wanted a Unix-like operating system to run on his home computer, and the only option available — MINIX — was too restricted to modify freely. So he wrote his own.

What he did not anticipate was that other people would want to contribute to it. Within a year, developers from across the world were sending him code. And this is where the first problem began — not a technical problem, but a coordination problem. How do you manage contributions from hundreds of people scattered across different time zones, working on different parts of the same codebase, when there is no shared system to track what changed, when, and why?

Torvalds' solution in those early years was brutally manual. Contributors would write their changes

as "patch" files — plain text documents describing exactly which lines of code were added, removed, or modified. They emailed these patches directly to Torvalds. He read them, judged them, and if he approved, applied them to his local copy of the source tree using a command-line tool called patch. When enough changes had accumulated, he would compress the entire source tree into a tarball — a single archived file — and release it as a new version of the kernel.

This system worked when Linux was small. It became a serious problem when Linux became important.

The Scaling Crisis of the Late 1990s

By the late 1990s, the Linux kernel had grown into critical infrastructure. It was running web servers, powering supercomputers, and forming the foundation of what would become the modern internet. The contributor base had grown from a handful of hobbyists to hundreds of professional developers employed by companies like IBM, Intel, and Red Hat — all submitting patches simultaneously.

Torvalds was drowning. His inbox received hundreds of patch emails per week. Merge conflicts — situations where two developers had independently modified the same section of code in incompatible ways — had to be resolved manually, one line at a time, by a single person. Code was occasionally lost. The pace of development was being throttled not by a lack of talent or ideas, but by the sheer impossibility of coordinating that many people through email and manual file editing.

Centralized version control systems existed. CVS (Concurrent Versions System) had been the standard since the late 1980s. Its successor, SVN (Subversion), was designed specifically to fix CVS's worst problems. But Torvalds had examined both and rejected them with characteristic bluntness. His objections were not superficial preferences. They were architectural:

CVS and SVN required every developer to connect to a central server to commit their work. If the server was unavailable — due to a network outage, a hardware failure, or simple geography — no one could commit code. For a globally distributed project with contributors in Europe, North America, and Asia working across different time zones, this was an unacceptable single point of failure.

More critically, branching and merging in CVS and SVN were slow and painful. Branching — creating a separate line of development to safely experiment with a new idea — was technically possible but so cumbersome that most developers avoided it. For the Linux kernel, which needed dozens of parallel experimental branches running simultaneously, this was not a minor inconvenience. It was a fundamental incompatibility with how kernel development actually worked.

Torvalds did not just dislike CVS. He considered it an actively harmful piece of software that made the problem worse by encouraging bad habits and false confidence in version history it could not reliably preserve.

2002 — The Devil's Bargain

With no suitable open-source alternative available, the Linux kernel team made a decision in 2002 that would haunt them for three years. They accepted a conditional free license from BitMover Inc. to use BitKeeper — a proprietary, closed-source distributed version control system built by Larry McVoy.

BitKeeper was genuinely different from anything that had come before. It was distributed, meaning every developer held a complete local copy of the entire repository with its full history. You could branch, commit, and merge entirely offline, only pushing your changes to others when you were ready. This matched exactly how the Linux kernel development community worked. The improvement in productivity was immediate and dramatic.

But the price was philosophical. The Linux kernel — the most important open-source project in the world, the embodiment of the argument that software should be free and transparent — was now dependent on a tool whose source code nobody outside BitMover could read, modify, or audit. Richard Stallman and the Free Software Foundation were vocal in their criticism. Torvalds was using a proprietary tool to build free software. The contradiction was obvious, and it made many developers deeply uneasy.

Larry McVoy's license came with a specific condition: the Linux community could use BitKeeper for free as long as they never attempted to reverse-engineer it or develop a competing product. For three years, an uncomfortable truce was held.

April 2005 — The Phone Call That Changed Everything

The truce ended in April 2005. Andrew Tridgell, the Australian developer who had created Samba — the open-source software that allows Linux systems to communicate with Windows networks — grew frustrated with BitKeeper's limitations and began analyzing its network protocol. His goal was to write a free client that could read BitKeeper repositories, improving interoperability for developers who did not want to use the proprietary tool directly.

McVoy considered this reverse-engineering a clear violation of the license agreement. He revoked the free license for the entire Linux kernel community, effective immediately.

The reaction in the Linux developer community was something between panic and fury. In a matter of days, thousands of developers lost access to the version control system that had been managing the Linux kernel for three years. There was no migration plan. There was no backup system. There was nothing.

Torvalds was faced with a choice: adopt one of the existing open-source alternatives, or build something new. He spent several days evaluating every available option. Subversion was out — it carried all the same centralized architecture problems he had rejected years earlier. Arch and Monotone were distributed but too slow and too limited for the scale of the Linux kernel. GNU's

own tools were not up to the task.

Torvalds had three non-negotiable requirements for whatever came next. First, it had to be fully distributed — no central server, every developer with a complete local copy of the history. Second, it had to be fast — applying a large patch set to the Linux kernel history had to happen in seconds, not minutes. Third, it had to guarantee cryptographic integrity — every single object in the repository had to be identified by a hash of its own contents, making any accidental or malicious corruption instantly detectable.

Nothing available met all three. So on April 3, 2005, Torvalds announced on the Linux kernel mailing list that he was going to write a new version control system. He was not sure it would work. He gave himself roughly a week to find out.

Ten Days in April

What followed has become one of the most remarkable episodes in open-source history. Working in intense, concentrated bursts, Torvalds designed and implemented the foundational core of Git in approximately ten days. The name was deliberately self-deprecating — "git" is British slang for a stupid, unpleasant person, and Torvalds applied it to himself with his characteristic dry humor, joking that he named all his projects after himself.

The architectural decision that made Git fundamentally different from everything before it was this: Git does not track files. It does not track changes to files. Git takes a complete **snapshot** of the entire project at every commit and stores that snapshot as a compressed object, identified by a SHA-1 cryptographic hash computed from the snapshot's complete contents. This means two things. First, the entire history of a project is a chain of cryptographically linked snapshots — you can verify at any point that nothing has been altered, because altering anything would change its hash, which would break the chain. Second, branching is essentially free — creating a new branch is just creating a new pointer to an existing snapshot, not copying files or diverging histories.

By June 2005 — less than two months after writing the first line — Git was stable and performant enough to host the Linux kernel itself. Torvalds handed maintenance of Git to Junio C Hamano, who has remained its principal maintainer to this day, and returned his focus to the kernel.

Why He Shared It Instead of Keeping It Proprietary

The answer to this question is almost too obvious, given the context. The entire crisis that produced Git happened precisely because a proprietary company could revoke access to a critical tool at will. Torvalds had just lived through the worst possible demonstration of why software infrastructure cannot be proprietary.

Beyond the immediate lesson, Torvalds had spent fifteen years building something in the open and watching it become more reliable, more secure, and more capable than anything built behind closed doors. He understood from direct experience that open development, with its thousands of

contributors finding bugs and proposing improvements, produces better software than any single team working in secret. Keeping Git proprietary would have contradicted everything the Linux project stood for and everything his own experience had taught him.

He licensed Git under the GPL v2 — the same license as the Linux kernel — ensuring that it would remain free and open forever, that no company could ever take it, improve it, and close it away from the community that needed it. Twenty years later, that decision looks like one of the most consequential acts of generosity in the history of computing.

A2. The license - What it actually says?

Git is licensed under the **GNU General Public License version 2.0 (GPL v2)**. To understand what this means, we have to examine the core tenets of free software which can be seen as follows.

The Four Freedoms of Free Software
The Free Software Foundation defines software as "free" if the end-user is granted four specific, irrevocable freedoms:

- **Freedom 0:** The freedom to run the program as you wish, for any purpose.
- **Freedom 1:** The freedom to study how the program works, and change it so it does your computing as you wish. (Access to the source code is a precondition for this).
- **Freedom 2:** The freedom to redistribute copies so you can help others.
- **Freedom 3:** The freedom to distribute copies of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes.

Git's GPL v2 license absolutely grants all four of these freedoms mentioned above. Anyone can download Git, read its C and shell source code, compile it, modify it to suit their own needs, and share those modifications in the community.

Can a company modify Git and sell it without sharing their changes?
The definitive answer is **No**. The GPL v2 is what is known as a "strong copyleft" license. It contains a viral clause: if you modify the GPL-licensed software and distribute that modified version (whether for free or for money), you *must* make the source code of your modifications available under the exact same GPL v2 license. A company cannot take Git, add a proprietary graphical user interface or performance upgrade to the core binary, and sell it as closed-source software. If they distribute the binary, they must distribute the modified source code. Thus ensuring that the software, and all its derivatives, remain permanently open.

MIT Licensing
The MIT License is a permissive software license originating at the Massachusetts Institute of Technology in the late 1980s. It stands as one of the most widely used permissive licenses in the open-source world. As a permissive license, it puts few restrictions on reuse and has high license compatibility. Unlike the GNU General Public License (GPL), it is a non-copyleft license, meaning it

imposes minimal restrictions on users. It grants complete freedom to use, copy, modify, and distribute the software, but crucially, it allows developers and corporations to take the code, improve upon it, and release their modified versions as closed-source or proprietary software without having to share those changes back with the community. This permissive nature makes the MIT License highly attractive for projects seeking maximum adoption and integration into commercial products, such as UI frameworks or small utility libraries, as it removes the fear of legal entanglement associated with strong copyleft clauses.

GPL vs. MIT Licensing: A Strategic Choice

The MIT License is fundamentally different. It is a "permissive" license. Here, permissive means Git provides an excellent example of "free as in freedom." It grants the same freedoms to use, modify, and distribute, but it *lacks* the copyleft clause. If Git were MIT-licensed, a corporation like Microsoft or Apple could take the code, improve it, and release a closed-source, proprietary version without ever giving their improvements back to the community.

If I were building a completely new software project today, my choice of license would depend entirely on my strategic goals.

- If my goal was **maximum widespread adoption and integration**—for example, a JavaScript UI framework or a small utility library—I would choose the **MIT License**. I would want corporations to use it without fear of legal entanglement, knowing that widespread use makes my tool the industry standard.
- However, if I were building a **foundational infrastructure tool** like an operating system, a database, or a version control system like Git, I would choose the **GPL**. This forces a reciprocal relationship: corporations can profit from my work, but they must contribute their bug fixes and feature additions back to the communal pool.

"Free as in Free Beer" vs "Free as in Freedom"

This distinction is crucial in open source.

- The term "free beer" (Gratis) refers specifically to cost. For instance, a proprietary mobile game qualifies as "free beer" because it can be downloaded at no expense; however, users are prohibited from viewing, altering, or studying its underlying source code.
- "Free speech" (Libre) means liberty. It represents the fundamental right of users who can control their own computing rather than being controlled by the software developer. In the context of Git, this liberty ensures that the global developer community is never beholden to a single entity for their version control needs. Unlike "free beer," which only addresses the lack of a price tag, "free speech" guarantees that the software remains a public good that can be audited, improved, and shared by anyone.

Git provides an excellent example of "free as in freedom." Yes, Git costs $0 (free beer). But more importantly, when GitHub (a multi-billion dollar platform) needed Git to scale to handle massive repositories, they didn't have to wait for Linus Torvalds to build those features. They exercised their *freedom* to modify the Git source code, optimize the packing algorithms, and submit those patches back to the Git mailing list. That is freedom.

A3. The ethics of open source

The idea of open source is not limited to just licenses or code sharing. It raises deeper questions about how knowledge, technology, and innovation should be shared in society. It challenges the balance between openness, profit, and responsibility.

Should all software be open source?

This is not a simple yes-or-no question. The answer depends on the purpose of the software and the context in which it is used.

Argument in Favor of Open Source

Supporters believe that software should be open because it promotes transparency and collective progress. When source code is accessible, developers can study, improve, and verify it. This reduces dependency on companies and avoids the “black box” problem, where users have no idea how software works.

Open source also reduces duplication of effort. Instead of multiple companies building similar tools separately, developers can collaborate and build better solutions together. This leads to faster innovation and shared growth.

Argument Against Open Source

On the other hand, not all software can realistically be open source. Some industries, such as healthcare, banking, or aerospace, require strict control, security, and accountability. In such cases, companies invest huge amounts of time and money, and they need legal protection to recover those investments.

If everything were open source, companies might lose motivation to invest in high-risk and expensive research. Proprietary systems also help maintain responsibility in critical systems where failure could cause serious harm.

Balanced View

In reality, both models are important. Open source works best in collaborative environments where transparency is valuable, while proprietary software is necessary in areas requiring strict control and accountability. A combination of both is what drives modern technology forward.

Is It Ethical for Companies to Profit from Open Source?

Many companies build successful businesses using open-source software. For example, platforms may use Git or Linux while generating revenue through services or interfaces.

Legally, this is allowed. However, ethically, it can be questioned. If companies only take from open-source communities without contributing back, it can feel unfair.

At the same time, many companies contribute actively to open-source projects. They invest resources, hire developers, and improve the software. This creates a balance where both the community and companies benefit.

So, the relationship is not one-sided—it is more like a partnership.

Although the debate about whether software should be open source or closed can be explored extensively, it is often a matter of context rather than a binary choice. Both models serve distinct needs depending on the situation. Open source thrives in collaborative environments where transparency and shared innovation are paramount, while closed source provides a framework for protecting massive R&D investments and ensuring strict liability in highly regulated industries like medical devices or banking. Ultimately, both methodologies are essential to the modern digital landscape, addressing different requirements for security, economic incentive, and public accessibility.

.

Standing on the Shoulders of Giants

The concept of "standing on the shoulders of giants" means building new ideas on top of existing knowledge. In software, open source makes this possible.

Developers do not need to start from scratch. For example, Git itself uses existing technologies like networking libraries and cryptographic functions. This allows developers to focus on innovation rather than reinventing basic components.

Open source reduces the effort required to begin a project. A student or developer can use existing frameworks and tools to build advanced systems quickly. This accelerates progress and encourages creativity.

Part B: Linux Footprint (Unit 2)

To analyze Git in a practical way, I examined how it is installed and used on a standard Ubuntu Linux system (24.04 LTS). This helped me understand how Git integrates with the Linux environment and how it behaves at the system level.

Installation Methods Across Platforms

Git is a cross-platform tool available for all major operating systems. Below are the standard methods to install and maintain Git on Windows, macOS, and Linux.

- **Windows**: On Windows, the most common method is using the Git for Windows installer (BASH, GUI, and Shell integration). Users can download the executable from the official website and follow the wizard. Alternatively, Windows users can use the winget package manager: ```winget install --id Git.Git -e --source winget.```
- **MacOS**: MacOS users can install Git by simply running `git --version` in the terminal; if not present, it will prompt to install the Xcode Command Line Tools. For a more up-to-date version, the Homebrew package manager is recommended: `brew install git`.
- **Linux (Ubuntu/Debian)**:Git is deeply integrated into almost all Linux distributions. On Debian/Ubuntu systems, the primary installation method is via the package manager:
  ```sudo apt update && sudo apt install git```
  For users requiring the absolute latest features or custom configurations, Git can be compiled directly from source. This involves downloading the tarball from kernel.org and running:
  ```make configure && ./configure --prefix=/usr && make all && sudo make install```

Key Directories (Linux)

After installation, Git does not exist as a single file but is distributed across different locations in the Linux file system according to standard conventions.

- The main executable is located in `/usr/bin/git`
- System-wide configuration files are stored in `/etc/gitconfig`
- User-specific configurations are stored in `~/.gitconfig`
- Git repositories are created in user-defined project directories

This distribution follows the Linux filesystem hierarchy and ensures proper organization, security, and accessibility.

POSIX guidelines:

Git follows standard Linux (POSIX) filesystem conventions, meaning its components are distributed in an organized manner rather than stored in a single location.

- **Binaries:**
  The main Git command is available at `/usr/bin/git`. However, Git is not just one program—it consists of many smaller helper tools such as `git-commit` and `git-merge`. These supporting binaries are usually stored in directories like `/usr/libexec/git-core/`.
- **Configuration:**
  Git maintains configuration at multiple levels:
  - System-wide settings (for all users) are stored in `/etc/gitconfig`
  - User-specific settings (such as username and email) are stored in `~/.gitconfig` or `~/.config/git/config`
  - Repository-specific configuration is stored inside the `.git/config` file within each project
- **Data Storage:**
  Git does not rely on a centralized database. Instead, it stores all project data locally. The entire version history, commits, and objects are stored inside a hidden `.git` directory in the root of each repository.

User and Group

Unlike services such as web servers or databases, Git does not run continuously in the background under a dedicated system account. It operates only when a user executes a command.

For example, when I run a Git command, it runs under my user account, and all files created (such as objects inside `.git/objects`) belong to that user.

This behavior is important for security. Since Git runs with user-level permissions, it cannot modify system-critical files unless explicitly executed with elevated privileges (like root access).

Service Management

Git is not managed as a system service. It is a command-line tool that runs only when invoked.

This means:

- It is not controlled using tools like `systemctl`
- It does not run continuously in the background

There is a component called `git daemon`, which allows repository access over a network. However, this method is rarely used today because secure communication through SSH is preferred.

Update Model

Updates and security patches for Git are handled by its core development community and then distributed through Linux package managers.

For example, in Ubuntu:

- Git updates are included in system updates
- Users can update Git using:
  sudo apt upgrade

Linux distributions ensure that updates are tested, verified, and signed before release. This provides reliability and security through trusted package management systems.

I verified these behaviors using commands such as `git config --list`, `ls -la .git`, and system update commands.

Part C: The FOSS Ecosystem (Units 3 & 4)

No open-source project exists in a vacuum. Git is deeply interwoven into the broader Free and Open Source Software (FOSS) ecosystem.

Dependencies
Git is written primarily in C and shell scripts, designed to be brutally fast and lightweight. However, it relies heavily on other foundational FOSS libraries:

- **zlib:** For compressing the objects (commits, trees, blobs) to save massive amounts of disk space.
- **libcurl:** For handling HTTP/HTTPS network transfers when pushing or pulling from remote servers.

- **OpenSSL / GnuTLS:** For establishing secure encrypted connections.
- **OpenSSH:** While not technically a library dependency, Git relies almost entirely on the SSH protocol to securely authenticate users and transfer data without needing its own proprietary authentication daemon.

What has Git enabled or inspired?
Git's impact is incalculable. It directly enabled the creation of **GitHub** (launched in 2008), which fundamentally changed software from a solitary engineering task into a social network. It spawned competitors and alternatives like **GitLab** and **Bitbucket**.
More importantly, Git inspired an entire movement known as **GitOps** and modern CI/CD (Continuous Integration / Continuous Deployment). Because Git tracks state perfectly, tools like Jenkins, GitHub Actions, and ArgoCD now use Git repositories as the "single source of truth" to automatically build, test, and deploy software to the cloud whenever code is merged.

Where does it fit in the LAMP stack?
Git is not formally part of the LAMP stack (Linux, Apache, MySQL, PHP/Python). However, it is the invisible layer that manages the stack. A modern developer does not FTP files directly to an Apache server anymore. Instead, they write their PHP code locally, commit it to Git, and push it to a repository. A deployment script then pulls the latest Git commit directly onto the LAMP server. Git acts as the transport and versioning layer for web development.

Community and Governance
The Git community is highly decentralized and extremely technical. It does not use GitHub for its own development; rather, true to its roots, the Git project itself is governed via a mailing list (git@vger.kernel.org). Developers submit patches as plain text emails.
The project is not controlled by a corporation. Following Linus Torvalds' departure from active Git maintenance, the project was handed over to **Junio C Hamano**, who has been the core maintainer since 2005. The governance is a meritocracy: patches are fiercely debated, heavily scrutinized, and only merged if they prove to be mathematically and architecturally sound.

Part D: Open Source vs Proprietary (Critical Analysis)

To understand Git's true value, it must be compared against a proprietary alternative. In enterprise environments, the closest proprietary alternative to Git is **Perforce (Helix Core)**.

Dimension	Open Source (Git)	Proprietary Alternative (Perforce Helix Core)
Cost	**Free.** $0 licensing fees, regardless of the number of users or repository size.	**Expensive.** Charges high per-user, per-year licensing fees after a small free tier.

Dimension	Open Source (Git)	Proprietary Alternative (Perforce Helix Core)
Security (Code Auditing)	**High.** Millions of developers can inspect the C code for vulnerabilities. Hashes guarantee integrity.	**Dependent on vendor.** Users cannot audit the source code. Must trust Perforce's internal security team.
Support and Reliability	**Community-driven.** Support comes from StackOverflow, forums, and documentation. Very high reliability.	**Corporate SLAs.** Guaranteed response times, dedicated account managers, and formal technical support.
Freedom to Modify	**Absolute.** Anyone can fork Git, change its algorithms, or build custom front-ends without asking permission.	**Zero.** The software is a closed binary. Modifications are legally prohibited and technically impossible.
Community vs Corporate	Governed by an open mailing list. Features are added based on global community consensus and need.	Features are dictated by a product roadmap managed by executives, often prioritizing high-paying enterprise clients.

Verdict

When comparing Git to proprietary solutions like Perforce, the dominance of open source becomes overwhelmingly clear for standard software development. Git's decentralized nature fundamentally aligns with how modern developers work: offline capable, branch-heavy, and experimental. The zero-cost barrier to entry allowed it to become the universal language of version control, meaning almost every graduating computer science student already knows how to use it, drastically reducing onboarding time for companies.

However, it is important to acknowledge that Git is not perfect for *every* situation. Proprietary tools like Perforce still maintain a strong grip on the video game industry because Git struggles natively with massive, gigabyte-sized binary files (like 3D models and textures), whereas Perforce was designed to stream large binaries centrally. Yet, even here, the open-source community responded by building Git LFS (Large File Storage) to bridge the gap. Given everything I have researched, I would unequivocally recommend Git for real-world deployment in 99% of software scenarios. It is robust, mathematically sound, and represents the absolute best of what open-source collaboration can achieve. I look forward to using it daily and contributing back to the tools that surround it.

4. Shell Script Tasks

Script 1 — System Identity Report

```bash
#!/bin/bash

# Script 1: System Identity Report
# --- Variables ---
STUDENT_NAME="[Your Name]"
SOFTWARE_CHOICE="Git"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE_TIME=$(date '+%Y-%m-%d %H:%M:%S')

# Attempt to get the OS distribution name safely
if [ -f /etc/os-release ]; then
   # Source the os-release file to get the PRETTY_NAME variable
   . /etc/os-release
   DISTRO_NAME=$PRETTY_NAME
else
   DISTRO_NAME="Unknown Linux Distribution"
fi

# --- Display ---
echo "=================================================="
echo "          The Open Source Audit                   "
echo "=================================================="
echo "Student : $STUDENT_NAME"
echo "Audit   :
$SOFTWARE_CHOICE"
echo "                                                  "
echo "System  : $DISTRO_NAME"
echo "Kernel : $KERNEL"
echo  "Date  :  $DATE_TIME"
echo  "User  :  $USER_NAME"
echo "Uptime : $UPTIME"
echo "                                                  "
echo "License Note: The Linux kernel running this system"
echo "is proudly protected by the GPL v2 open-source"
```

```
echo "license, guaranteeing your freedom to study it."
echo "==================================================="

# --- Report Documentation ---
# Purpose: This script provides an overview of the active Linux
environment. It
# generates a tailored banner by retrieving the distribution name, kernel
release,
# current user, and system uptime.
# Technical Logic: Command   substitution `$()` is employed to store the
results of
# utilities like `uname -r`  and `uptime -p`. To avoid hardcoding values,
the script
# uses conditional logic to source `/etc/os-release` for accurate
distribution
# identification, paired with `echo` commands for a structured user
interface.
```

OUTPUT:

```
  Roll No   : 24BCY10236
  Auditing  : Git

 ---------------------------------------------
  OS        : Ubuntu 22.04.3 LTS
  Kernel    : 5.15.x-microsoft-standard-WSL2
  Arch      : x86_64
  Hostname  : AMMAR-DELL
  User      : ammar
  Uptime    : up X minutes
  Date/Time : Tue Mar 31 16:32:00 IST 2026
 ---------------------------------------------

  Audit SW  : git version 2.xx.x

 ---------------------------------------------
  License   : The Linux kernel powering this machine
              is licensed under the GNU GPL v2,
              ensuring freedom to use, study,
              modify, and distribute the software.
 =============================================
```

Script 2 — FOSS Package Inspector

```
#!/bin/bash

# Script 2: FOSS Package Inspector
# We will check the software chosen for the audit: git

PACKAGE="git"

echo "Inspecting package: $PACKAGE..."
echo "                                        "

# Check if package is installed.
# Using dpkg -s for Debian/Ubuntu systems. (If on RedHat, use rpm -q
$PACKAGE)
if dpkg -s "$PACKAGE" &> /dev/null; then
   echo "[SUCCESS] $PACKAGE is installed on this system."

   # Extract version and maintainer info using pipe and grep
   dpkg -s "$PACKAGE" | grep -E '^Version|^Maintainer|^Architecture'

   echo "                                        "
   echo "Philosophical Summary:"

 # Case statement to print a one-line philosophy based on the package name
   case "$PACKAGE" in
       git)
           echo "Git: The distributed engine of global collaboration, born
from necessity."
           ;;
       apache2|httpd)
             echo "Apache: The web server that essentially built the open
internet."
           ;;
       mysql|mysql-server)
            echo "MySQL: Open source data storage at the heart of millions
of apps."
```

```
            ;;
        vlc)
                echo "VLC: The media player that plays anything,   built by
students for freedom."
            ;;
        *)
            echo "A valuable piece of the open-source puzzle."
            ;;
    esac

else
    echo "[WARNING] $PACKAGE is NOT installed."
    echo "You can install it using: sudo apt install $PACKAGE"
fi
echo "====================================="

else
    echo "[WARNING] $PACKAGE could not be found."
    echo "To proceed, install it via: sudo apt install $PACKAGE"
fi
echo "====================================="
```

```
# --- Report Analysis and Technical Explanation ---
# Functionality: This script serves as a verification tool to confirm the
local installation of Git.
# Upon detection, it retrieves specific version metadata and utilizes a
case-based logic
# to provide a unique philosophical interpretation of the software.
#
# Implemented Logic: The implementation relies on an `if-then-else`
structure that uses
# standard output redirection (`&> /dev/null`) to non-intrusively evaluate
the return
# status of `dpkg -s`. Data flows through pipes (`|`) into `grep -E`,
utilizing regular
# expressions to isolate relevant package details. Finally, a `case`
statement provides
```

```
# a scalable and clean method for managing various software identity
outputs.
```

Output:

```
❯ ./script2_inspector.sh
Inspecting package: git...
--------------------------------------
[SUCCESS] git is installed on this system.
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 1:2.43.0-1ubuntu7.3
--------------------------------------
Philosophical Summary:
Git: The distributed engine of global collaboration, born from necessity.
======================================
```

Script 3 — Disk and Permission Auditor

```
#!/bin/bash

# Script 3: Disk and Permission Auditor
# Array of standard system directories, plus the local user's git config
directory
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "$HOME/.git")

echo "=========================================================="
echo "          Directory Space and Permission Audit
" echo
"==========================================================="
printf "%-25s | %-12s | %-10s\n" "Directory" "Permissions"
"Size" echo "                                                    "

# Loop through each item in the array
for DIR in "${DIRS[@]}"; do
   # Check if the directory actually exists on this system
   if [ -d "$DIR" ]; then
            # Use stat to get human-readable permissions string (e.g.,
drwxr-xr-x)
```

```
        # Using stat is often cleaner than ls -ld | awk
        PERMS=$(stat -c "%A" "$DIR" 2>/dev/null)

            # Get the size using du -sh, suppress errors for restricted
folders, cut first field
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

            # If size is empty (due to severe permission denial), set to
'Denied'
        if [ -z "$SIZE" ]; then
            SIZE="Denied"
        fi

        # Print formatted output
        printf "%-25s | %-12s | %-10s\n" "$DIR" "$PERMS"
    "$SIZE" else
        printf "%-25s | %-12s | %-10s\n" "$DIR" "NOT FOUND" "N/A"
    fi
done
echo "============================================================"

# --- Technical Breakdown for the Audit Report ---
# Objective: This script evaluates the status and security of key Linux
filesystem
# locations by scanning an array of essential directories (such as
Git-related paths).
# It verifies their presence, checks r/w/x permissions, and calculates
disk consumption.
# Methodology: A Bash array holds the target paths, which are then handled
through
# a `for DIR in "${DIRS[@]}"` loop. The script uses the `[ -d ]` operator
for validation,
# utilizes command substitution with `du`, `stat`, and `cut`, and silences
permission
# errors by routing `stderr` to `2>/dev/null` for a polished output.
```

Output:

```
❯ ./script3_auditor.sh
============================================================
          Directory Space and Permission Audit
============================================================
Directory                  | Permissions  | Size
------------------------------------------------------------
/etc                       | drwxr-xr-x   | 19M
/var/log                   | drwxrwxr-x   | 808M
/home                      | drwxr-xr-x   | 98G
/usr/bin                   | drwxr-xr-x   | 561M
/tmp                       | drwxrwxrwt   | 664K
/home/jay/.git             | NOT FOUND    | N/A
============================================================
```

Script 4 — Log File Analyzer

```bash
#!/bin/bash

# Script 4: Log File Analyzer
# Command-line arguments
LOGFILE=$1
# Default keyword is 'error' if second argument is not provided
KEYWORD=${2:-"error"}
COUNT=0

# Ensure a log file was actually provided
if [ -z "$LOGFILE" ]; then
   echo "Usage: ./script4_analyzer.sh <path_to_log_file> [keyword]"
   echo "Example: ./script4_analyzer.sh /var/log/syslog warning"
   exit 1
fi

# Do-while style loop to handle missing or unreadable files
while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
   echo "Error: File '$LOGFILE' not found or not readable."
   read -p "Please enter a valid, readable log file path (or 'q' to quit):
" NEW_PATH
```

```
   if [ "$NEW_PATH" == "q" ]; then
       echo "Exiting analyzer."
       exit 1
   fi
   LOGFILE="$NEW_PATH"
done

echo "Analyzing '$LOGFILE' for the keyword '$KEYWORD'..."
echo "                                                          "

# Enable case-insensitive matching natively in Bash for the loop
shopt -s nocasematch

# Read the file line by line safely using while-read loop
# Note: We use native Bash string matching here instead of 'echo $LINE |
grep'
# because spawning a 'grep' process for 100,000+ lines in syslog causes
extreme hanging.
while IFS= read -r LINE; do
   # Check if the line contains the keyword (using built-in bash globbing)
   if [[ "$LINE" == *"$KEYWORD"* ]]; then
       COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"

# Revert case-insensitive matching back to default
shopt -u nocasematch

echo "SUMMARY: Keyword '$KEYWORD' was found $COUNT times."

# If matches were found, print the last 5 matching lines as requested in
TODO
if [ $COUNT -gt 0 ]; then
   echo "                                                          "
   echo "Last 5 matching lines:"
   # Use grep to find all matches efficiently at once, then pipe to tail
   grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

```
fi

# --- Technical Analysis for the Audit Report ---
# Functionality: This script operates as a basic log monitoring utility.
It accepts a
# file path and a search term as input, performs a secure line-by-line
traversal to
# tabulate keyword matches, and generates a summary report including the
final
# captured entries.
#
# Core Concepts: The implementation leverages positional parameters `$1`
and `$2`,
# utilizing default variable expansion (`${2:-"error"}`) for the keyword.
For
# efficient and safe text processing, it employs a `while IFS= read -r
LINE` loop.
#
#  Performance  Note:  Native  Bash  pattern matching (`[[ "$LINE" ==
*"$KEYWORD"* ]]`)
# is used alongside `nocasematch`. This approach avoids the significant
overhead
# of initializing thousands of external `grep` processes, preventing
execution
# timeouts when auditing high-volume files like syslog.
```

Output:

```
❯ ./script4_analyzer.sh /var/log/syslog google
Analyzing '/var/log/syslog' for the keyword 'google'...
-----------------------------------------------------
SUMMARY: Keyword 'google' was found 4641 times.
-----------------------------------------------------
Last 5 matching lines:
2026-03-24T20:54:14.325387+05:30 jay-Vostro-3590 google-chrome.desktop[9104]: [9090:9090:0324/205414.321289:ERROR:extensions/browser/api/web_requ
est/extension_web_request_event_router.cc:1919] AddEventListener: 0x99c00a6c6c0 extension_id: pkehgijcmpdhfbdbbnkijodmdjhbjlgp extension_name: Pr
ivacy Badger event_name: webRequest.onBeforeSendHeaders sub_event_name: webRequest.onBeforeSendHeaders/s2 render_process_id: 908 web_view_instanc
e_id: 0 worker_thread_id: 1 service_worker_version_id: 682
2026-03-24T20:54:14.325434+05:30 jay-Vostro-3590 google-chrome.desktop[9104]: [9090:9090:0324/205414.322224:ERROR:extensions/browser/api/web_requ
est/extension_web_request_event_router.cc:1919] AddEventListener: 0x99c00a6c6c0 extension_id: pkehgijcmpdhfbdbbnkijodmdjhbjlgp extension_name: Pr
ivacy Badger event_name: webRequest.onResponseStarted sub_event_name: webRequest.onResponseStarted/s3 render_process_id: 908 web_view_instance_id
: 0 worker_thread_id: 1 service_worker_version_id: 682
2026-03-24T20:55:24.194908+05:30 jay-Vostro-3590 google-chrome.desktop[9104]: [9090:9090:0324/205524.194557:ERROR:extensions/browser/api/web_requ
est/extension_web_request_event_router.cc:1919] AddEventListener: 0x99c00a6c6c0 extension_id: pkehgijcmpdhfbdbbnkijodmdjhbjlgp extension_name: Pr
ivacy Badger event_name: webRequest.onBeforeRequest sub_event_name: webRequest.onBeforeRequest/s1 render_process_id: 911 web_view_instance_id: 0
worker_thread_id: 1 service_worker_version_id: 682
2026-03-24T20:55:24.195393+05:30 jay-Vostro-3590 google-chrome.desktop[9104]: [9090:9090:0324/205524.195207:ERROR:extensions/browser/api/web_requ
est/extension_web_request_event_router.cc:1919] AddEventListener: 0x99c00a6c6c0 extension_id: pkehgijcmpdhfbdbbnkijodmdjhbjlgp extension_name: Pr
ivacy Badger event_name: webRequest.onBeforeSendHeaders sub_event_name: webRequest.onBeforeSendHeaders/s2 render_process_id: 911 web_view_instanc
e_id: 0 worker_thread_id: 1 service_worker_version_id: 682
2026-03-24T20:55:24.195976+05:30 jay-Vostro-3590 google-chrome.desktop[9104]: [9090:9090:0324/205524.195915:ERROR:extensions/browser/api/web_requ
est/extension_web_request_event_router.cc:1919] AddEventListener: 0x99c00a6c6c0 extension_id: pkehgijcmpdhfbdbbnkijodmdjhbjlgp extension_name: Pr
ivacy Badger event_name: webRequest.onResponseStarted sub_event_name: webRequest.onResponseStarted/s3 render_process_id: 911 web_view_instance_id
: 0 worker_thread_id: 1 service_worker_version_id: 682
```

Script 5 — The Open Source Manifesto Generator

```bash
#!/bin/bash

# Script 5: The Open Source Manifesto Generator
# Author: [Your Name]
# Description: Interactively prompts the user to generate a personalized
text file.
# Concepts: read for input, file redirection, variables, aliasing concept.

# Aliasing concept demonstration (Note: aliases usually don't expand in
scripts
# unless shopt -s expand_aliases is set, but we demonstrate the concept
here)
# alias create_file='touch'

echo "==============================================="
echo "      The Open Source Manifesto Generator
" echo
"==============================================="
echo "Answer three brief questions to generate your manifesto."
echo ""

# Get user input interactively
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you in computing? "
```

```
FREEDOM
read -p "3. Name one thing you would like to build and share freely: " BUILD

# Fetch current date and format it
DATE=$(date '+%d %B %Y')
# Generate a unique output filename based on the current user
OUTPUT="manifesto_$(whoami).txt"

echo ""
echo "Drafting manifesto..."
sleep 1 # Adds a nice dramatic pause

# Compose the paragraph and write to the file using redirection (>)
# Using a Here-Doc for cleaner multi-line text generation
cat <<EOF > "$OUTPUT"

A DEVELOPER's MANIFESTO--------------------------------
Date: $DATE
Author: $(whoami)

------------------------------------------------------------
I believe that software shapes human reality.
Every single day, I rely on community-built tools like $TOOL to do my work.
Because of this, I understand that in the realm of computing, freedom
fundamentally means $FREEDOM.

I do not want to just consume; I intend to contribute.
Standing on the shoulders of the open-source giants who came before me,
my goal is to one day build $BUILD and release it back to the community,
sharing it freely so that others may learn from it, improve it, and build
upon it.

The code is open. The future is open.

EOF
------------------------------------------------------------
```

```
echo "[SUCCESS] Manifesto generated and saved to: $OUTPUT"
echo "Here is what you wrote:"
echo "============================================"
# Read the file back to the terminal
cat "$OUTPUT"
echo "============================================"

# --- Technical Summary for Script 5 ---
# Functional Overview: This script is designed to generate a custom "Open
Source Manifesto".
# By prompting the user with three philosophical inquiries, it assembles
their responses into
# an inspiring, unified text block that is saved to a uniquely named file
and displayed.

# Applied Techniques: To capture interactive data, I implemented the `read
-p` command.
# Rather than using repetitive `echo >>` calls, I employed a "Here-Doc" (`cat
<<EOF > file`),
# providing a more efficient method for managing multiline file redirection
(`>`) and string
# formatting. Additionally, the `date` command provides dynamic
timestamps.
```

Output:

```
❯ ./script5_manifesto.sh
==============================================
   The Open Source Manifesto Generator
==============================================
Answer three brief questions to generate your manifesto.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you in computing? Happiness
3. Name one thing you would like to build and share freely: AI

Drafting manifesto...
[SUCCESS] Manifesto generated and saved to: manifesto_jay.txt
Here is what you wrote:
==============================================
--------------------------------------------------
A DEVELOPER's MANIFESTO
Date: 24 March 2026
Author: jay
--------------------------------------------------

I believe that software shapes human reality.
Every single day, I rely on community-built tools like Git to do my work.
Because of this, I understand that in the realm of computing, freedom fundamentally means Happiness.


I do not want to just consume; I intend to contribute.
Standing on the shoulders of the open-source giants who came before me,
my goal is to one day build AI and release it back to the community,
sharing it freely so that others may learn from it, improve it, and build upon it.

The code is open. The future is open.
--------------------------------------------------
==============================================
```